Azure Cloud Cost Chargeback & FinOps Platform

Building cost visibility, fair-share allocation and showback for a multi-tenant Azure estate

FinOps / Cloud Cost Engineering - Portfolio Case Study

Databricks • PySpark • Azure Data Factory • ADLS Gen2 • PostgreSQL • GitHub Actions

This document describes an enterprise cloud cost chargeback platform. Organisation-specific identifiers, credentials, account names and financial figures have been redacted or generalised; the architecture, data model and business logic are described as implemented.

Contents

1. Executive Summary

2. The Problem

3. FinOps Framing

3.1 Capability Coverage

4. Architecture at a Glance

4.1 Logical Architecture

4.2 Technology Choices and Why

5. Stage 1 - Ingestion and Normalisation

5.1 Extractors

5.2 Normalisation Work

6. Stage 2 - The Mark-Up Engine

6.1 Savings Categories Measured

6.2 The Cost-Centre-Owned Reservation Carve-Out

7. Stage 3 - Fair-Share Credit Allocation

7.1 Allocation Mechanics

7.2 Eligibility Is a Business Rule, Not a Lookup

8. Stage 4 - Cost Centre Resolution

8.1 The Inheritance Waterfall

8.2 Precedence Order (highest authority first)

8.3 Why Provenance Matters More Than the Answer

9. Stage 5 - Human-in-the-Loop Finance Override

10. Stage 6 - Reconciliation to the Actual Invoice

10.1 True-Up Formula

10.2 Supporting Controls

11. Stage 7 - Aggregation, Tax Allocation and Publication

11.1 The Two Serving Grains

11.2 Analytical Enrichment

11.3 Tax Allocation

11.4 Publication

12. Data Quality and Trust Engineering

12.1 Controls

13. Engineering Practices

14. Outcomes

15. What I Would Do Differently

1. Executive Summary

Public cloud bills arrive as a single, undifferentiated invoice. For a large enterprise running thousands of Azure resources across multiple tenants and subscriptions, that invoice answers the question "how much did we spend?" but never the question that finance and engineering leaders actually ask: "who spent it, on what, and was it worth it?"

This platform closes that gap. It is a production Databricks and PySpark data pipeline that ingests the raw Azure Enterprise Agreement amortized cost export every day, enriches it with resource metadata, rate cards, VM licensing state and Kubernetes topology, resolves every single billing line to an owning cost centre through an eleven-level inheritance waterfall, redistributes reservation and licensing savings fairly across the teams that generated them, reconciles the result back to the actual invoice to the cent, allocates indirect taxes, and publishes cost-centre-level showback and budget-tracking datasets to a reporting warehouse.

The outcome is the foundational capability every FinOps practice depends on: trustworthy, daily, resource-level cost attribution that both engineering and finance agree on. It moves the organisation from a monthly, aggregate, after-the-fact invoice to a daily, attributable, defensible chargeback position.

2. The Problem

- The Azure invoice is organised by billing construct (subscription, meter, reservation), not by the organisational construct finance runs on (cost centre, segment, business unit).
- Tagging is incomplete and inconsistent. A large share of resources carry no CostCenter tag at all, carry a malformed one (leading zeros, trailing newlines, mixed case), or carry a cost centre that no longer exists in the finance master.
- Reserved Instances, Savings Plans and Hybrid Benefit licensing create a fairness problem: the savings are earned centrally but consumed by individual workloads. Naive allocation either gives the whole discount to whoever happened to land on a reserved VM, or hides it entirely at the enterprise level.
- Amortized cost and actual invoiced cost never match exactly. Finance will not accept a chargeback number that does not reconcile to the invoice.
- Indirect and non-recoverable taxes are a real cost that must be recovered, but they are invoiced in aggregate with no resource-level attribution.
- Derived resources (managed disks, LTR backup storage, VMSS instances, AKS node resource groups, Databricks managed resource groups) are billed under system-generated identities that no human ever tags.
- Nothing downstream - budgets, forecasts, optimisation targets, unit economics - can be built until attribution is solved first.

3. FinOps Framing

The platform maps directly onto the FinOps Foundation capability model, and sits squarely in the Inform phase, which is the prerequisite for everything else.

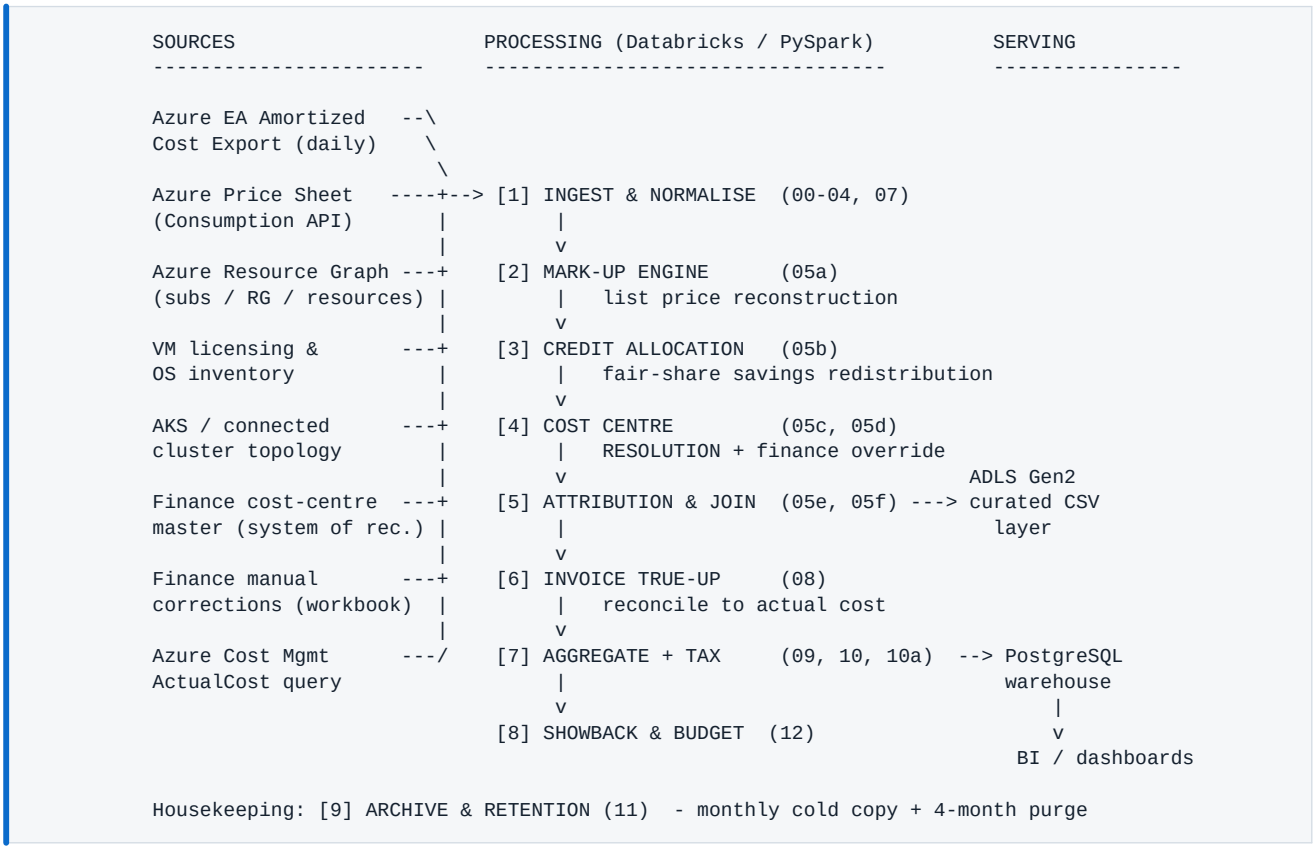
3.1 Capability Coverage

FinOps Phase	Capability	How this platform delivers it
Inform	Data Ingestion & Normalisation	Daily pull of the EA amortized export, rate card, resource graph inventory, VM licensing state and AKS topology into a normalised Spark schema.
Inform	Allocation	Eleven-level cost-centre inheritance waterfall plus a finance override layer; 100% of spend lands on an owning cost centre.
Inform	Reporting & Analytics	Category-grain and resource-grain fact tables published to a partitioned Postgres warehouse for BI consumption.
Inform	Chargeback & Showback	Cost-centre-level daily charges with month-to-date accumulation and budget-versus-actual percentage tracking.
Inform	Data Quality & Reconciliation	Row-count deltas on every join, cost-sum assertions across every aggregation, and a hard reconciliation to the actual invoice.
Optimize	Rate Optimisation visibility	Reservation, Savings Plan, BYOL and SQL Hybrid Benefit savings are measured per line and surfaced as explicit credit columns.
Operate	Budget Management	Daily actuals joined to the approved monthly forecast per cost centre, expressed as percent-of-budget consumed.
Operate	Governance & Accountability	Every allocation decision records its provenance; a human-in-the-loop finance correction layer is versioned and backed up.

4. Architecture at a Glance

The platform is a batch pipeline orchestrated by Azure Data Factory, executing Databricks notebooks written in PySpark and pandas, over a data lake on ADLS Gen2, serving a PostgreSQL reporting warehouse.

4.1 Logical Architecture



4.2 Technology Choices and Why

Layer	Technology	Rationale
Orchestration	Azure Data Factory	Already the enterprise scheduler; passes run dates and mode flags into notebooks as widgets, enabling the same code to serve daily and monthly runs.
Compute	Databricks + PySpark	The amortized export is tens of millions of rows per month; the credit allocation and cost-centre waterfall are wide-join problems that Spark handles natively.
Light transforms	pandas	Used where datasets are small (rate card, AKS topology, cost-centre daily report) and the pivot/report shape is awkward in Spark.
Storage	ADLS Gen2 (abfss)	Immutable, date-partitioned, cheap audit trail of every intermediate stage; any day of any month can be re-derived and explained.
Serving	PostgreSQL, monthly declarative partitions	BI-friendly, and month partitions make the idempotent delete-and-reload pattern cheap and safe.
Secrets	Key Vault-backed Databricks secret scope	No credentials in code; environment separation is achieved by swapping the scope, not by branching logic.
CI/CD	GitHub Actions + Databricks Repos	Promotion from the development branch to the production branch triggers a workspace Git-folder sync - deployment is a branch merge.

5. Stage 1 - Ingestion and Normalisation

Five independent extractors run daily or monthly, each landing an immutable, date-partitioned snapshot. Keeping extraction separate from transformation means a source outage never corrupts downstream state, and any historical month can be reprocessed from the lake without re-querying Azure.

5.1 Extractors

Stage	What it pulls	Cadence	Why it is needed
Metadata	Every subscription, resource group and resource with its tags, across both Azure tenants, via Azure Resource Manager	Daily, plus a month-to-date latest-wins consolidation	Supplies the inheritance chain for untagged resources. The daily snapshot matters because resources are deleted mid-month and their tags vanish with them.
Price Sheet	The negotiated EA rate card for the billing period	Monthly, idempotent (exits early if already present)	Provides list and negotiated unit prices - the baseline against which reservation and licensing savings are measured.
VM Inventory	OS type, OS name, and licence type for every VM, plus NIC and IP detail, via Resource Graph KQL	Daily full snapshot	The licence type field is the Azure Hybrid Benefit signal; without it, BYOL savings cannot be identified or redistributed.
Cluster Topology	All managed, Arc-connected and provisioned Kubernetes clusters, flattened to one row per agent pool, plus a node-count rollup	Daily full snapshot	Container platform costs land in system-generated node resource groups; topology is what maps them back to an owning cluster and team.
Actual Cost	Invoiced ActualCost totals by publisher, subscription and meter category, via Cost Management query	Monthly	The reconciliation target. This is the number finance considers true.

5.2 Normalisation Work

- The Azure additional-info and tag payloads arrive as embedded JSON strings; they are parsed against an explicit schema and exploded into first-class columns (VM name, vCPU count, reservation order and reservation id, target resource id).
- Cost-centre tags are aggressively cleaned before use: leading and trailing whitespace stripped, leading zeros stripped, embedded newlines removed, and lower-cased. In practice this single step rescues a meaningful share of spend that would otherwise fall through to an unknown bucket.

- Every cleaned cost centre is validated against the finance cost-centre master. A tag that looks valid but does not exist in the system of record is worse than no tag, because it silently charges a real team the wrong amount - so it is reset and re-derived through the waterfall.
- Known platform data-quality defects are corrected in-flight. For example, SQL Database line items report the server name rather than the database name, so the resource name is re-derived from the resource id path.
- Rate card parsing handles commas embedded inside parenthesised meter names (tier ranges such as 250,001-500,000) that would otherwise break naive CSV splitting.
- Marketplace charges are separated from first-party Azure charges, because they follow a different pricing and reconciliation path.
- Schemas are serialised to the lake as JSON contracts between stages, so a downstream notebook reads exactly the shape the upstream notebook wrote.

6. Stage 2 - The Mark-Up Engine

This is the first of the two commercial-logic stages, and it answers a deceptively simple question: what would this line item have cost at list price, with no commitments and no licensing benefits?

That counterfactual is necessary because discounts are earned centrally but consumed locally. A team that happens to land on a reserved VM should not get a silent 40% discount while an identical team on an on-demand VM pays full price. Reconstructing the list price creates a level playing field; the savings are then redistributed deliberately in the next stage.

6.1 Savings Categories Measured

Benefit	Savings formula (per line)	Mark-up applied
Reserved Instance / Savings Plan	(unit price - effective price) x quantity	Line is re-priced at the undiscounted unit price.
Windows Server BYOL	(full Windows VM rate - unit price) x quantity	Line is re-priced at the equivalent licence-included Windows VM rate, derived from the rate card by meter name, region and promo/basic tier.
SQL Server Azure Hybrid Benefit	equivalent licence price - software assurance cost, where the assurance cost is normalised vCore-hours x a per-vCore-hour rate	Measured as an explicit saving column; Enterprise edition is normalised at 4x the vCore weighting of Standard.
Unused commitments	Unused reservation and unused savings plan charges are zeroed on the line	Their cost is absorbed into the savings pool and redistributed, so waste is shared rather than landing arbitrarily.

6.2 The Cost-Centre-Owned Reservation Carve-Out

Not all commitments are enterprise-funded. Some reservations are purchased directly by an individual cost centre, which encodes its identity in the reservation name using an agreed naming convention.

These lines are detected by that naming convention and flagged. Flagged lines are excluded from mark-up, excluded from contributing to the shared savings pool, and excluded from receiving redistributed credit - they are passed through at true cost, and the reservation name itself becomes an authoritative source of cost-centre attribution later in the waterfall. This is a good example of a business rule that cannot be inferred from the billing data alone and has to be encoded deliberately: if a team paid for its own commitment, it keeps its own discount.

7. Stage 3 - Fair-Share Credit Allocation

With every line re-priced to list, the savings earned across the estate form a pool. The allocation stage gives that pool back to the workloads that actually generated it, pro rata by marked-up spend, within service boundaries.

Allocating within service boundaries rather than globally is the key design decision. A SQL reservation discount belongs to SQL consumers, not to storage consumers who happened to be on the same invoice. Each eligible service family therefore gets its own pool, its own denominator, and its own distribution.

7.1 Allocation Mechanics

```
For each benefit pool P (Redis Premium, SQL Database, SQL Managed Instance,
Virtual Machines, Synapse dedicated pools, MySQL, PostgreSQL, App Service plans,
and Windows/SQL BYOL):

    pool_total(P)     = SUM( saving )        over lines earning benefit P
    pool_basis(P)     = SUM( marked_up_cost) over lines ELIGIBLE for benefit P

    credit(line, P)   = marked_up_cost(line) / pool_basis(P) * pool_total(P)

Final charge per line:

    chargeable = marked_up_cost - SUM over P of credit(line, P)

Invariant: SUM(chargeable) across the estate == SUM(actual consumption cost).
Mark-up and credit net to zero at the enterprise level - the exercise
redistributes, it does not inflate.
```

7.2 Eligibility Is a Business Rule, Not a Lookup

- Eligibility is defined per service using meter category and meter sub-category, and encodes which SKUs a given reservation type can actually apply to - for example only Gen5 compute tiers for SQL Database, only Premium tier for Redis, only Premium and Isolated plans for App Service, and only the VM series families that reservations cover.
- Virtual machine BYOL eligibility cannot be read from the bill at all. It is resolved by joining the billing line to the live VM inventory and reasoning over OS type, licence type and OS name - a Linux VM with no service info is excluded, a Windows VM carrying a server licence type is included, and client-OS images are specifically excluded.
- Savings Plan usage is treated as broadly eligible regardless of VM series, because savings plans apply across families where reservations do not.
- Getting eligibility wrong does not change the enterprise total, but it systematically over- or under-charges specific teams - which is precisely the failure mode that destroys trust in a chargeback model.

8. Stage 4 - Cost Centre Resolution

This is the heart of the platform and the hardest problem it solves. Every billing line must end up owned by exactly one cost centre, but only a fraction of resources are correctly tagged at source.

Rather than resolving ownership on the billing-line grain, where the same resource appears many times per month, the pipeline builds a separate resource-grain ownership map once per month and then joins it back to the charges. This makes the expensive waterfall cheap to compute, easy to audit, and reusable.

8.1 The Inheritance Waterfall

Candidate cost centres are evaluated in strict precedence order. The first confident source wins, and critically, the source of the decision is recorded alongside the value - every allocated dollar can be traced back to why it landed where it did.

8.2 Precedence Order (highest authority first)

#	Source	Rule
1	Finance override	An explicit correction supplied by finance through a controlled workbook. Always wins, even over a valid tag.
2	Resource tag	The resource's own CostCenter tag, cleaned and validated against the finance master.
3	Same-resource sibling tag	Another billing line for the same resource id carries a valid tag; propagate it to the untagged lines.
4	Resource inventory tag	The tag on the resource as seen in the Resource Graph inventory snapshot (catches resources deleted mid-month).
5	Resource group tag	Inherit from the parent resource group.
6	Subscription tag	Inherit from the parent subscription.
7	Platform job tag	For a managed application platform, the workload type tag distinguishes tenant workloads from platform overhead, each routed to its own cost centre.
8	Dependent-resource inheritance	Long-term-retention backup storage and managed-instance backup storage have no tags of their own; a composite key reconstructs the parent database or instance and the charge follows the parent.
9	Cost-centre-owned reservation	The owning cost centre encoded in the reservation name.
10	Prior-month carry-forward	The resource's resolved cost centre from last month's map - stability for resources whose tags lapsed.
11	Regional default	A geography-based catch-all so that nothing is ever left unallocated.

8.3 Why Provenance Matters More Than the Answer

The resource map carries every candidate value side by side - the tag from the bill, the sibling value, the resource value, the resource-group value, the subscription value, the dependent-parent value, last month's value, the interim machine-derived answer, the finance-processed answer, and the final answer - plus a source

label.

This turns disputes into five-minute conversations instead of five-day investigations. When a team challenges a charge, the answer is not "the pipeline says so"; it is "this resource had no tag, its resource group had no tag, and its subscription is owned by you - here is the lineage." In practice, that auditability is what makes a chargeback model politically survivable, and it also turns the allocation report into a tagging-hygiene scorecard: the distribution of source labels is a direct measure of tagging maturity over time.

9. Stage 5 - Human-in-the-Loop Finance Override

No automated rule set is ever complete. Reorganisations, project transfers, shared-service splits and genuine mis-tagging all require human judgement. Rather than fighting that reality, the pipeline makes it a first-class, controlled stage.

Finance maintains a correction workbook exposing three override mechanisms, each more specific than the last: a blanket cost-centre-to-cost-centre remap for reorganisations; a resource-level override keyed on resource id; and a VM-plus-disk-LUN override for the specific case of managed disks attached to shared virtual machines, where the disk must be charged to a different owner than the VM.

The overrides are applied twice by design - once to the resource ownership map, and again directly to the charge rows after attribution. The second pass is not redundant: it catches lines whose cost centre came from a valid tag on the bill and therefore never consulted the map. Every workbook version is backed up to the lake before use, so an override can always be reproduced and explained after the fact, and month-end re-runs can deliberately replay the prior month's corrections.

10. Stage 6 - Reconciliation to the Actual Invoice

Amortized cost and invoiced cost never agree exactly. Amortization spreads commitment purchases across their term using Azure's own model; rounding, credits, unassigned charges and timing differences all introduce drift. A chargeback number that does not reconcile to the invoice will be rejected by finance, correctly.

The true-up stage closes that gap deterministically. Invoiced actual cost is pulled per meter category, cleaned (marketplace excluded, unassigned reservation charges reattributed to compute), and compared against the amortized total for the same category. The residual difference is then spread back across the individual lines in proportion to their chargeable value.

10.1 True-Up Formula

```
Per meter category C:

    delta(C) = actual_invoiced_cost(C) - amortized_chargeable_cost(C)

Per line L in category C:

    adjustment(L) = chargeable(L) / amortized_chargeable_cost(C) * delta(C)
    final(L)      = chargeable(L) + adjustment(L)

Guarantee:  SUM(final) per category == invoiced cost per category

The adjustment is retained as its own column rather than folded silently into
the price, so any team can see exactly how much of its charge is invoice
reconciliation versus actual consumption.
```

10.2 Supporting Controls

- A dedicated late-charges run mode re-processes a closed month after corrections, taking a versioned backup of the prior output before overwriting.
- Manual adjustment lines can be appended for genuinely out-of-band charges, bypassing the spread calculation by design.
- The finance-facing export renames internal columns back to the terminology finance already uses, separating pre-adjustment charge, invoice adjustment, and final charge as three distinct, explainable figures.

11. Stage 7 - Aggregation, Tax Allocation and Publication

Two parallel aggregations serve two different audiences from the same reconciled detail, and a third stage layers tax on top before publication.

11.1 The Two Serving Grains

Dataset	Grain	Primary consumer and question answered
Category-level fact	Month x cost centre x subscription x environment x service taxonomy x meter x pricing model x charge type x cluster type	Finance and FinOps leadership. 'What are we spending on, by service, and how is it trending?'
Resource-level fact	The above plus resource id, resource name and cluster name	Engineering teams. 'Which of my specific resources is driving my cost?' This is also the grain optimisation recommendations attach to.
Container platform fact	Workspace, cluster name, cluster id and cluster-level cost-centre tag	Data platform teams. Isolates managed analytics spend, with ephemeral job-cluster names normalised so that cost rolls up to the job rather than fragmenting across thousands of one-off cluster ids.
Cost-centre daily showback	Cost centre x day, wide and unpivoted	Budget owners. 'How much have I burned this month and what percent of my approved budget is that?'

11.2 Analytical Enrichment

- Environment classification derived from subscription naming convention (production, non-production, sandbox, proof-of-technology, other), applied in precedence order so that the most specific match wins. This is what makes the classic FinOps question - how much are we spending on non-production? - answerable in a single filter.
- Subscription generation classification (modern landing-zone versus legacy estate), which lets cloud-migration programmes measure their own progress in cost terms.
- Workload type derived from the managed-by relationship: static resources that a human created, versus dynamic resources a platform created on their behalf, further classified into analytics, container, data warehouse, backup, scale-set and governance platforms. Dynamic spend is the spend engineers cannot see in the portal and therefore never optimise.
- A curated service taxonomy (service type, service name, service code) maintained by the finance team maps raw Azure meter categories onto the organisation's own service catalogue. The pipeline deliberately fails hard when a new meter category appears without a mapping, rather than silently bucketing new Azure services into an 'other' category where they would go unnoticed.
- An override file allows environment and subscription classification to be corrected for subscriptions that do not follow the naming convention.

11.3 Tax Allocation

Indirect taxes are invoiced in aggregate and are only partially recoverable, and the recoverable proportion differs by business segment for jurisdictional reasons. Allocating tax evenly across spend would therefore over-charge some segments and under-charge others.

The pipeline models this in three steps. First, the total tax pool for the month is established - either a finance-agreed locked figure for closed months, or a default percentage of total chargeback for open months. Second, each business segment's taxable base is computed as its spend multiplied by its segment-specific recovery rate, and the tax pool is split across segments in proportion to those taxable bases. Third, each segment's tax share is pushed down to individual rows in proportion to their final price, producing a per-resource and per-category tax amount.

The locked-figure mechanism is a small but important piece of financial engineering: once a month is closed and agreed with finance, its tax allocation must never silently change because an upstream input was re-processed.

11.4 Publication

- Both fact tables are published to PostgreSQL using monthly declarative partitions, with a delete-then-append pattern scoped to the month being processed. Re-running any day of any month is safe and idempotent - a non-negotiable property for a pipeline that runs unsupervised every day.
- Partitions are created on demand for each new month, so no manual DDL is required at month boundaries.
- The daily showback dataset is published in both wide form (one column per day of month, for finance-style reporting) and unpivoted long form (one row per cost centre per day, for time-series visualisation).

- Budget context is joined from the approved monthly forecast, taking the latest approved version per cost centre, so showback is always expressed as a percentage of an agreed number rather than as a bare dollar figure.

12. Data Quality and Trust Engineering

A chargeback platform's product is not data - it is trust. One unexplained number costs more credibility than a hundred correct ones earn. The pipeline is therefore built with controls at every seam.

12.1 Controls

Control	Where	What it protects against
Join cardinality checks	Row counts captured before and after every single enrichment join, with the delta surfaced	Silent row multiplication from a non-unique lookup key - the single most common way a cost pipeline inflates spend.
Lookup key de-duplication	Every reference and override dataset is explicitly de-duplicated on its join key before use	Fan-out at the source, rather than detecting it after the fact.
Cost-sum assertions	Totals compared before and after each aggregation; the pipeline raises and fails on mismatch	Aggregation errors, type coercion losses and dropped rows.
Reference-data validation	Every cost centre validated against the finance system of record	Charging a real team under a stale or fat-fingered cost centre.
Fail-fast on unmapped taxonomy	Hard exception when a new meter category has no service mapping	New Azure services quietly disappearing into an unclassified bucket.
Sentinel-file dependency checks	Downstream notebooks verify upstream outputs exist and exit gracefully if not	Processing a partial day and publishing an understated bill.
Immutable staged outputs	Every stage writes a date-partitioned artefact to the lake	Inability to answer 'what did we believe on the 14th?' after a restatement.
Explicit schema contracts	Schemas serialised and passed between stages rather than inferred	Silent type drift and column reordering between pipeline stages.
Invoice reconciliation	Final true-up to invoiced actual cost per meter category	The gap between what the pipeline says and what the business actually paid.

13. Engineering Practices

- Configuration, secrets, authentication and shared schemas are centralised in a single common module, imported by every stage - credentials never appear in pipeline code.
- Secrets are resolved from a Key Vault-backed secret scope; environment separation is achieved by swapping the scope rather than by conditional logic, which keeps development and production code byte-identical.
- Authentication to Azure uses a service principal with client-credentials flow, fanned out across multiple tenants for the estate-wide inventory pulls.
- All date and mode parameters are injected as notebook widgets by the orchestrator, so the same artefact serves daily incremental runs, monthly close runs, and late-charge re-runs without code changes.
- Deployment is branch-driven: a merge to the production branch triggers a GitHub Actions workflow that synchronises the Databricks workspace Git folder to that branch. Promotion is a pull request, with full review history.
- A month-end housekeeping stage archives the raw bill and consolidated metadata to a cold container and enforces a rolling retention window on the hot lake, controlling storage cost - the FinOps platform applying FinOps to itself.
- Dependency and vulnerability scanning is wired into the repository alongside the application inventory registration, so the pipeline is governed like any other production service.

14. Outcomes

- Complete attribution: every dollar of first-party Azure spend is assigned to an owning cost centre, with no residual unallocated bucket, every single day.
- Fair-share economics: commitment and licensing savings are redistributed to the workloads that earned them, so teams see a price that reflects the enterprise's true negotiated position rather than an accident of scheduling.
- Finance-grade accuracy: chargeback reconciles to the invoice by meter category, with the reconciliation adjustment itself exposed as a separate, explainable line.
- Daily rather than monthly feedback: budget owners see consumption and percent-of-budget the next morning, not three weeks after month end - which is the difference between correcting an overspend and reporting one.
- Dual-audience reporting from a single reconciled source: finance gets service-level aggregates, engineering gets resource-level detail, and the two always tie out.
- Auditability as a feature: full provenance on every allocation decision converts chargeback disputes from investigations into lookups.
- A tagging-maturity feedback loop: the distribution of allocation sources is a direct, quantitative measure of tagging hygiene, and makes the business case for improving it self-evident.
- A foundation for the next FinOps phases: with trustworthy attribution in place, rightsizing, commitment planning, anomaly detection and unit-economics work all become tractable.

15. What I Would Do Differently

- Move the curated layer from CSV on the lake to Delta Lake, gaining ACID transactions, schema enforcement, time travel and merge-based upserts - which would retire the hand-rolled schema-contract and backup-copy mechanisms entirely.
- Replace the serialised-schema-passing pattern with versioned contracts in a schema registry or a dbt-style model layer, so schema drift between stages becomes a build-time failure rather than a runtime surprise.
- Formalise the scattered reconciliation print statements into a declarative data-quality framework with persisted results, so data quality becomes a trend that can be dashboarded rather than a log line someone has to read.
- Externalise the business rules - eligibility sets, recovery rates, regional defaults, platform cost centres - into versioned configuration rather than code conditionals, so finance can change a rate without a deployment.
- Extend coverage from first-party Azure to marketplace and other cloud providers behind a single allocation model, since the waterfall and credit logic generalise cleanly.
- Layer anomaly detection on the daily cost-centre series, turning the showback dataset from a passive report into an active alerting surface.
- Push attribution upstream: the clearest long-term win is not better inheritance logic, but tag policy enforcement at resource-creation time, with the waterfall retained as a safety net rather than relied on as the primary mechanism.